

PPO-Based Employee Scheduling (REINFORCE-full-problem)

This notebook frames the employee scheduling problem as a Reinforcement Learning task and solves it using **Proximal Policy Optimization (PPO)** with an MLP Actor-Critic network.

Problem Formulation

The goal is to assign shifts to employees across an entire year while respecting hard constraints:

- **223 workdays** per employee per year
- Max **5 consecutive** workdays
- Max **22 special day** (holiday/weekend) assignments
- Employees can only be assigned to **their own teams**
- No backward shift transitions (e.g., Afternoon -> Morning the next day)
- Respect **vacation days**
- Meet **minimum staffing demand** per shift/team each day

The problem is modeled as a **Gym environment** where the RL agent makes one decision per (employee, day) pair sequentially.

```
def __init__(self, data_dir: str =
".../.../data/problems/SMARTASK_SIMPLE_2025"):
    super().__init__()
    base = Path(data_dir)

    with open(base / "problem.json") as f:
        prob = json.load(f)

    self.num_days = prob["temporalScope"]["numDays"]
    self.year = prob["temporalScope"]["year"]
    employees = prob["employees"]["simple"]
    self.num_employees = len(employees)
    self.employee_teams = [set(emp.get("teams", [])) for emp in
employees]

    vac_df = pd.read_csv(base / "vacations.csv", header=None)
    self.vac_mask = vac_df.iloc[:, 1:].values.astype(bool)

    dem_df = pd.read_csv(base / "demand.csv")
    dem_df["date"] = pd.to_datetime(dem_df["date"])
    start_ts = pd.Timestamp(f"{self.year}-01-01")
    dem_df["day_idx"] = (dem_df["date"] - start_ts).dt.days

    self.min_demand = np.zeros((self.num_days, 2, 2), dtype=int)
    _shift_idx = {"M": 0, "T": 1}
    _team_idx = {"A": 0, "B": 1}
    for _, row in dem_df.iterrows():
```

```

        d = int(row["day_idx"])
        s = _shift_idx[row["shift"]]
        t = _team_idx[row["team"]]
        self.min_demand[d, s, t] = int(row["minimum"])

    self.special_days = _build_special_days(self.year, self.num_days)

    self.max_days_per_year = 223
    self.max_consecutive_days = 5
    self.special_days_cap = 22

    self.total_steps = self.num_employees * self.num_days
    self.num_features = 2

    self.observation_space = gym.spaces.Box(
        low=-1.0, high=10.0,
        shape=(self.total_steps, self.num_features + self.NUM_ACTIONS),
        dtype=np.float32,
    )
    self.action_space = gym.spaces.Discrete(self.NUM_ACTIONS)
    self.reset()

```

Environment (ScheduleEnv)

State Representation

The state is a **day-major matrix** of shape `(num_employees x num_days, num_features + num_actions)`. All employees for day 0 come first, then day 1, etc.:

Row	emp	day	action columns (one-hot)
0	0	0	0 0 0 0 0
1	1	0	0 0 0 0 0
2	2	0	0 0 0 0 0
-- day boundary --			
3	0	1	0 0 0 0 0
...			

The agent processes the schedule **one day at a time**, assigning shifts to all employees on day 0 before moving to day 1. This is natural because shift assignments for a given day are interdependent (they must meet that day's demand).

```

def _build_initial_matrix(self):
    """Sequential calendar-order day-major matrix."""
    matrix = np.zeros(
        (self.total_steps, self.num_features + self.NUM_ACTIONS),
        dtype=np.float32
    )
    self.emp_day_to_row = {}

```

```
row = 0
for day in range(self.num_days):
    for emp in range(self.num_employees):
        matrix[row, 0] = emp
        matrix[row, 1] = day
        self.emp_day_to_row[(emp, day)] = row
        row += 1
return matrix
```

Action Space

5 discrete actions per step:

Action	Meaning
0	Rest (day off)
1	Morning shift, Team A
2	Tarde shift, Team A
3	Morning shift, Team B
4	Tarde shift, Team B

Illegal actions are masked (set to probability 0) before sampling, enforcing hard constraints at the network level.

Reward Function

The reward balances **demand satisfaction** and **workload pacing**:

Scenario	Reward	Rationale
Work & demand still needs filling	+2.0	Directly satisfies a constraint
Work & demand already met	-1.0	Wastes the employee's 223-day budget
Work & too far ahead of pace	-0.3 to -2.0	Prevents burning budget too early
Rest & ahead of pace	+0.25 to +1.0	Conserves budget for later
Rest & behind pace, demand unmet, can work	-1.5	Missed opportunity to fill demand
End of day, all demand met	+3.0	Strong bonus for fully covered day
End of day, demand unmet	-2.0 x shortfall	Heaviest penalty, per missing person

Pacing is tracked by comparing each employee's days worked against the ideal linear pace (~0.611 days worked per calendar day = 223/365).

```

def _calculate_reward(self, emp_id, day_id, action):
    remaining_days_in_year = self.num_days - day_id - 1
    remaining_budget = self.max_days_per_year -
self.days_worked[emp_id]
    ideal_pace = self.max_days_per_year / self.num_days # ~0.611
    reward = 0.0

    # How many days ahead/behind the linear schedule
    expected_worked = (day_id + 1) * ideal_pace
    ahead_by = self.days_worked[emp_id] - expected_worked

    if action != 0:
        shift, team = ACTION_TO_SHIFT_TEAM[action]
        s = 0 if shift == "M" else 1
        t = 0 if team == "A" else 1
        cov = self.daily_coverage[day_id, s, t]
        mn = self.min_demand[day_id, s, t]

        if cov <= mn:
            reward += 2.0
        else:
            reward -= 1.0

        # Proportional pacing penalty when ahead of schedule
        if ahead_by > 3:
            reward -= min(2.0, 0.3 * (ahead_by - 3))

    else:
        # REST
        day_shortfall = np.maximum(0, self.min_demand[day_id] -
self.daily_coverage[day_id]).sum()

        if ahead_by > 2:
            # Ahead of pace: resting is good, save budget for later
            reward += min(1.0, 0.25 * (ahead_by - 2))
        elif day_shortfall > 0 and remaining_budget > 0:
            # Behind pace with unmet demand: resting is bad
            if remaining_days_in_year > 0:
                pace = remaining_budget / remaining_days_in_year
                if pace >= ideal_pace:
                    reward -= 1.5

    # End-of-day signal after last employee on this day
    is_last_emp_today = (
        self.current_step >= self.total_steps or
        int(self.state[self.current_step, 1]) != day_id
    )
    if is_last_emp_today:
        day_shortfall = np.maximum(0, self.min_demand[day_id] -
self.daily_coverage[day_id]).sum()
        if day_shortfall == 0:
            reward += 3.0
        else:

```

```
        reward -= day_shortfall * 2.0

    return reward

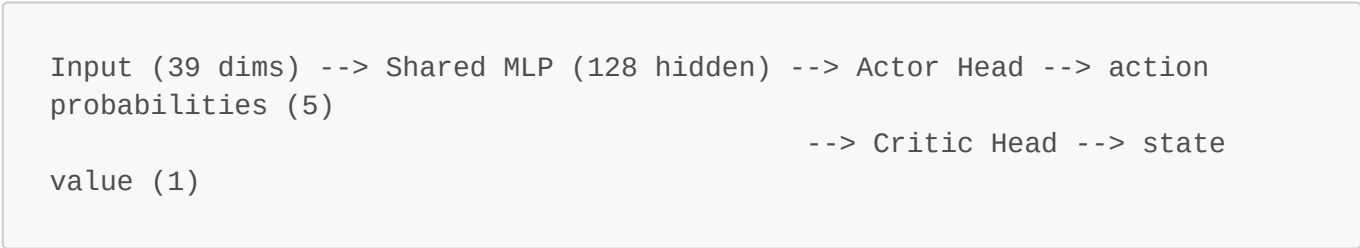
def _calculate_final_reward(self):
    reward = 0.0
    total_shortfall = 0
    for day in range(self.num_days):
        for s in range(2):
            for t in range(2):
                shortfall = self.min_demand[day, s, t] -
self.daily_coverage[day, s, t]
                if shortfall > 0:
                    total_shortfall += shortfall
    reward -= total_shortfall * 5.0

    for emp in range(self.num_employees):
        miss = self.max_days_per_year - self.days_worked[emp]
        if miss > 0:
            reward -= miss * 3.0

    if total_shortfall == 0:
        reward += 500.0
    return reward
```

Network Architecture (MLPActorCritic)

An MLP Actor-Critic with a **shared trunk** and two separate heads:



Input Features (39 dimensions)

Component	Dims	Description
Employee embedding	16	Learned vector per employee (captures team membership, individual traits)
Day projection	16	Static day features (normalized demand, special day flag, position in year) projected from 6 to 16 dims
Coverage gaps	4	min_demand - current_coverage per shift/team (M-A, T-A, M-B, T-B)
Dynamic features	3	days_worked (/ 223), consecutive_days (/ 5), pace_ratio (remaining_budget / remaining_calendar_days)

Actor Head

Outputs 5 logits, one per action. Illegal actions are masked to `-inf` before softmax, producing a valid probability distribution over legal actions only.

Critic Head

Outputs a single scalar estimating the expected future reward from the current state. Used to compute advantages during training.

```
class MLPActorCritic(nn.Module):
    """
    MLP Actor-Critic for PPO + GAE.
    Input per step: emp_emb + day_feat + cov_gap(4) + dyn_feats(3).
    dyn_feats = [days_worked, consecutive_days, pace_ratio].
    pace_ratio = remaining_budget / remaining_calendar_days.
    """
    DAY_FEAT_DIM = 6

    def __init__(self, num_employees, day_features, num_actions=5,
emb_dim=16, hidden_dim=128):
        super().__init__()
        self.num_days = day_features.shape[0]

        self.emp_emb = nn.Embedding(num_employees, emb_dim)
        self.register_buffer("day_feat_table", day_features)
        self.day_proj = nn.Linear(self.DAY_FEAT_DIM, emb_dim)

        in_dim = emb_dim * 2 + 4 + 3

        self.shared = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.Tanh(),
        )
        self.actor_head = nn.Sequential(
            nn.Linear(hidden_dim, 64),
            nn.Tanh(),
            nn.Linear(64, num_actions),
        )
        self.critic_head = nn.Sequential(
            nn.Linear(hidden_dim, 64),
            nn.Tanh(),
            nn.Linear(64, 1),
        )

    def _process_inputs(self, emp_ids, day_ids, cov_gaps, dyn_feats):
        e_emb = self.emp_emb(emp_ids)
        d_emb = self.day_proj(self.day_feat_table[day_ids])

        norm_dyn = dyn_feats.clone()
        norm_dyn[..., 0] /= 223.0
        norm_dyn[..., 1] /= 5.0
```

```

        return torch.cat([e_emb, d_emb, cov_gaps, norm_dyn], dim=-1)

def forward(self, emp_ids, day_ids, cov_gaps, dyn_feats, action_masks):
    x = self._process_inputs(emp_ids, day_ids, cov_gaps, dyn_feats)
    shared_feat = self.shared(x)

    values = self.critic_head(shared_feat).squeeze(-1)

    logits = self.actor_head(shared_feat)
    masks_bool = torch.as_tensor(action_masks, dtype=torch.bool)
    if not masks_bool.any():
        masks_bool[..., 0] = True
    logits = logits.masked_fill(~masks_bool, float("-inf"))
    probs = F.softmax(logits, dim=-1)

    return probs, values

```

Change from GNN to MLP

The Initial GNN Approach: The environment was mapped as a bipartite heterograph of employees and days.

The Problem (Over-smoothing): Because the initial state matrix included all possible (employee, day) combinations, it formed a complete bipartite graph. The SAGEConv("mean") layers averaged everything together, destroying structural variance and producing uniform, uninformative "mush" embeddings.

The Solution: Fixing the GNN would have required expensive dynamic edge-building at every step or complex attention mechanisms. Instead, it was changed to a highly efficient MLP Actor-Critic model.

Why it Works: By feeding the MLP carefully engineered features—learnable ID embeddings, real-time coverage gaps, and dynamic pacing signals—the network successfully learns the exact same relational patterns with a fraction of the computational overhead.

Training Loop (PPO)

Training alternates between two phases:

Phase 1: Collect Trajectory

The agent plays through an **entire episode** (all days x all employees) using the current policy, with no gradients. Every step records:

- State inputs (employee, day, coverage gaps, dynamic features, action mask)
- Action taken (sampled stochastically from the policy)
- Log-probability of that action under the current policy
- Reward received
- Critic's value estimate

```

def collect_trajectory(env, model):
    model.eval()

```

```

traj = Trajectory()
env.reset()
terminated = truncated = False

with torch.no_grad():
    while not (terminated or truncated):
        emp_id = int(env.state[env.current_step, 0])
        day_id = int(env.state[env.current_step, 1])

        cov_gap = [
            env.min_demand[day_id, 0, 0] - env.daily_coverage[day_id,
0, 0],
            env.min_demand[day_id, 1, 0] - env.daily_coverage[day_id,
1, 0],
            env.min_demand[day_id, 0, 1] - env.daily_coverage[day_id,
0, 1],
            env.min_demand[day_id, 1, 1] - env.daily_coverage[day_id,
1, 1],
        ]

        remaining_budget = env.max_days_per_year -
env.days_worked[emp_id]
        remaining_cal = max(1, env.num_days - day_id)
        pace_ratio = remaining_budget / remaining_cal

        dyn_feat = [env.days_worked[emp_id],
env.consecutive_days[emp_id], pace_ratio]
        action_mask = env.get_action_mask()

        emp_id_t = torch.tensor([emp_id], dtype=torch.long)
        day_id_t = torch.tensor([day_id], dtype=torch.long)
        cov_gap_t = torch.tensor([cov_gap], dtype=torch.float32)
        dyn_feat_t = torch.tensor([dyn_feat], dtype=torch.float32)
        mask_t = torch.tensor(action_mask.tolist(),
dtype=torch.bool).unsqueeze(0)

        probs, values = model(emp_id_t, day_id_t, cov_gap_t,
dyn_feat_t, mask_t)

        dist = Categorical(probs=probs[0])
        action = dist.sample()

        _, reward, terminated, truncated, _ = env.step(action.item())

        traj.emp_ids.append(emp_id)
        traj.day_ids.append(day_id)
        traj.coverage_gaps.append(cov_gap_t[0])
        traj.dyn_emp_feats.append(dyn_feat_t[0])
        traj.action_masks.append(mask_t[0])
        traj.actions.append(action.item())
        traj.log_probs_old.append(dist.log_prob(action).item())
        traj.rewards.append(float(reward))
        traj.values.append(values[0].item())

```



```
return traj
```

Phase 2: Update Weights

The recorded trajectory is used to improve the network:

1. **Compute advantages** using Generalized Advantage Estimation (GAE, $\lambda=0.95$, $\gamma=1.0$)
2. **Train for K=4 epochs**, shuffling the trajectory into random mini-batches of 512
3. Each mini-batch update computes:
 - **Importance sampling ratio**: $r = \pi_{\text{new}}(a|s) / \pi_{\text{old}}(a|s)$ -- corrects for the fact that data was collected with older weights
 - **Clipped actor loss**: $\min(r * A, \text{clip}(r, 0.8, 1.2) * A)$ -- prevents the policy from changing too drastically
 - **Entropy bonus**: encourages exploration by rewarding spread-out action distributions
 - **Critic loss**: smooth L1 loss between predicted and actual returns

The schedule itself is never reused -- it only serves as training data. With improved weights, a new episode is collected and the cycle repeats.

```
def ppo_update(model, optimizer, batch, advantages, returns, clip_eps=0.2,
               value_coeff=0.5, entropy_coeff=0.01, K_epochs=4, mini_batch_size=512):
    T = batch["actions"].shape[0]
    losses = []
    track_entropy = []
    model.train()

    for _ in range(K_epochs):
        for index in BatchSampler(SubsetRandomSampler(range(T)),
                                  mini_batch_size, False):
            idx = torch.tensor(index)

            probs_new, values_new = model(
                batch["emp_ids"][idx],
                batch["day_ids"][idx],
                batch["coverage_gaps"][idx],
                batch["dyn_emp_feats"][idx],
                batch["action_masks"][idx],
            )

            dist_now = Categorical(probs=probs_new)
            a_logprob_now = dist_now.log_prob(batch["actions"][idx])
            dist_entropy = dist_now.entropy()

            ratios = torch.exp(a_logprob_now - batch["log_probs_old"][idx])
            adv = advantages[idx]
            surr1 = ratios * adv
            surr2 = torch.clamp(ratios, 1.0 - clip_eps, 1.0 + clip_eps) *
adv
            actor_loss = -torch.min(surr1, surr2).mean() - entropy_coeff *
```

```
dist_entropy.mean()

    critic_loss = F.smooth_l1_loss(values_new, returns[idx])

    loss = actor_loss + value_coeff * critic_loss

    track_entropy.append(dist_entropy.mean().item())

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 0.5)
    optimizer.step()
    losses.append(loss.item())

return float(np.mean(losses)), float(np.mean(track_entropy))
```

Key Hyperparameters

Parameter	Value	Purpose
Episodes	10,000	Total training episodes
Gamma	1.0	No discounting (entire year matters equally)
Lambda (GAE)	0.95	Bias-variance tradeoff for advantage estimation
Clip epsilon	0.2	PPO clipping range [0.8, 1.2]
K epochs	4	Reuse each trajectory 4 times
Mini-batch size	512	Random mini-batches for gradient updates
Learning rate	3e-4	Adam optimizer
Entropy coeff	0.02 -> 0.002	Decays over training (explore less as policy improves)

Why Mini-Batches Instead of Full Trajectory?

- **Memory:** computing gradients for all steps at once requires storing all intermediate activations
- **More frequent updates:** ~14 updates per epoch instead of 1, so later mini-batches benefit from earlier updates
- **Shuffling breaks correlation:** consecutive steps (same day) are correlated; random batches give more independent gradient estimates